一、LLM 能力边界拓展

1.大模型应用架构

大模型应用架构是指以大型语言模型（LLM）为核心引擎，构建完整、可落地的应用系统的整体技术框架。它将LLM从单纯的文本生成能力，扩展为支持复杂业务逻辑、外部工具集成、长期记忆管理以及自主执行任务的端到端解决方案。

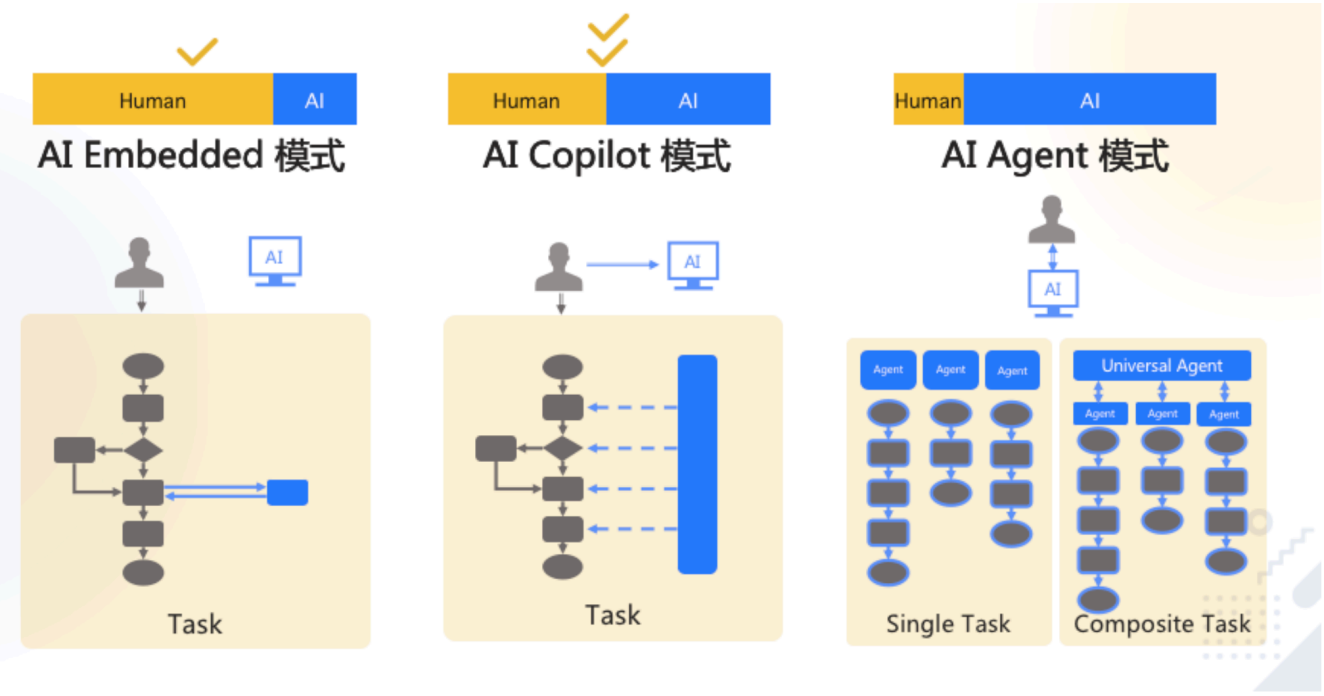
该架构的核心目标是将LLM的通用智能能力与具体业务场景深度融合，形成稳定、高效、安全的生产级应用。

1.1 典型模式

AI Embedded、AI Copilot 与 AI Agent 属于**大模型应用的三种典型模式**（也称为AI应用的三类架构或人机协同的三种范式）。

该分类以**AI自主程度与人类协作方式**为维度，将大模型在实际业务中的应用形态划分为三个递进层次：从被动嵌入到辅助协作，再到自主执行。这一分类是大模型应用架构中的核心框架之一，反映了AI从“功能增强”到“任务自动化”的演进路径。

这三种模式并非孤立存在，而是层层递进：AI Embedded 为基础形态，实现无感知的功能增强；AI Copilot 为过渡形态，实现人与AI的紧密协作；AI Agent 为高级形态，实现目标驱动的自主任务完成。

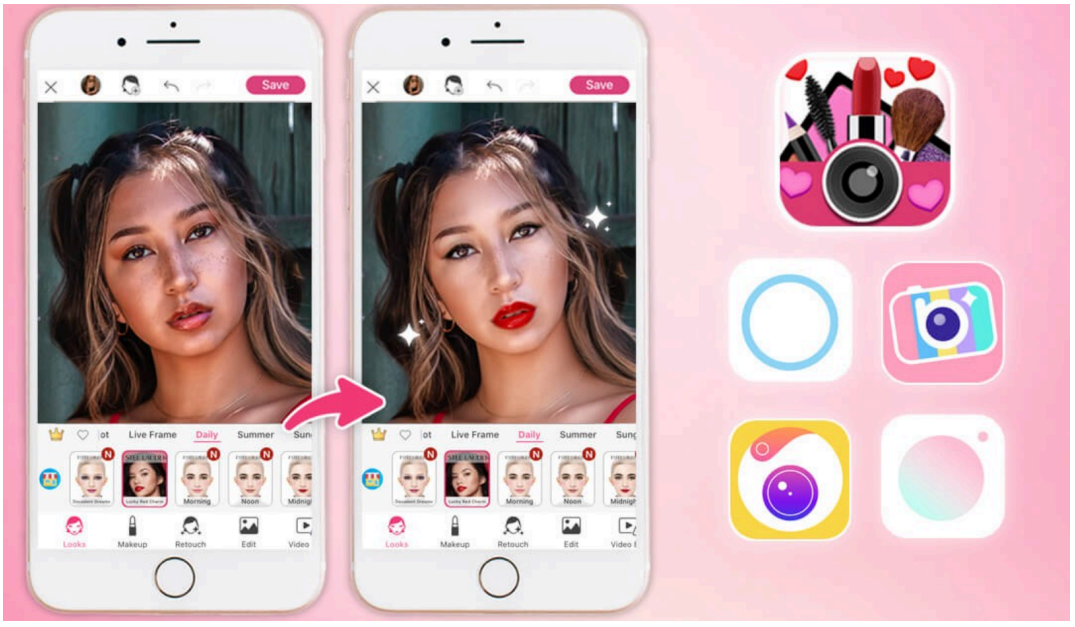


1.1.1 AI Embedded

（1）定义

AI Embedded 是人工智能最基础的应用形态，其核心在于将AI能力深度融合到现有产品、应用或硬件系统中，成为产品不可分割的组成部分。在这种模式下，用户几乎无法感知到AI的存在，AI像隐形引擎一样在后台自动运行，持续优化用户体验，而无需用户主动介入或调用。

这种模式的被动触发特性意味着AI功能仅在特定场景下被系统自动激活，例如当用户打开相机时，AI自动完成美颜、对焦和场景识别，无需用户额外操作。这种设计极大提升了产品的流畅性和易用性，让AI真正"消失"在交互界面背后。AI Embedded 通常采用传统机器学习模型结合轻量级LLM实现，技术实现相对简单，部署成本较低，因此广泛应用于消费级产品中。



（2）常见应用

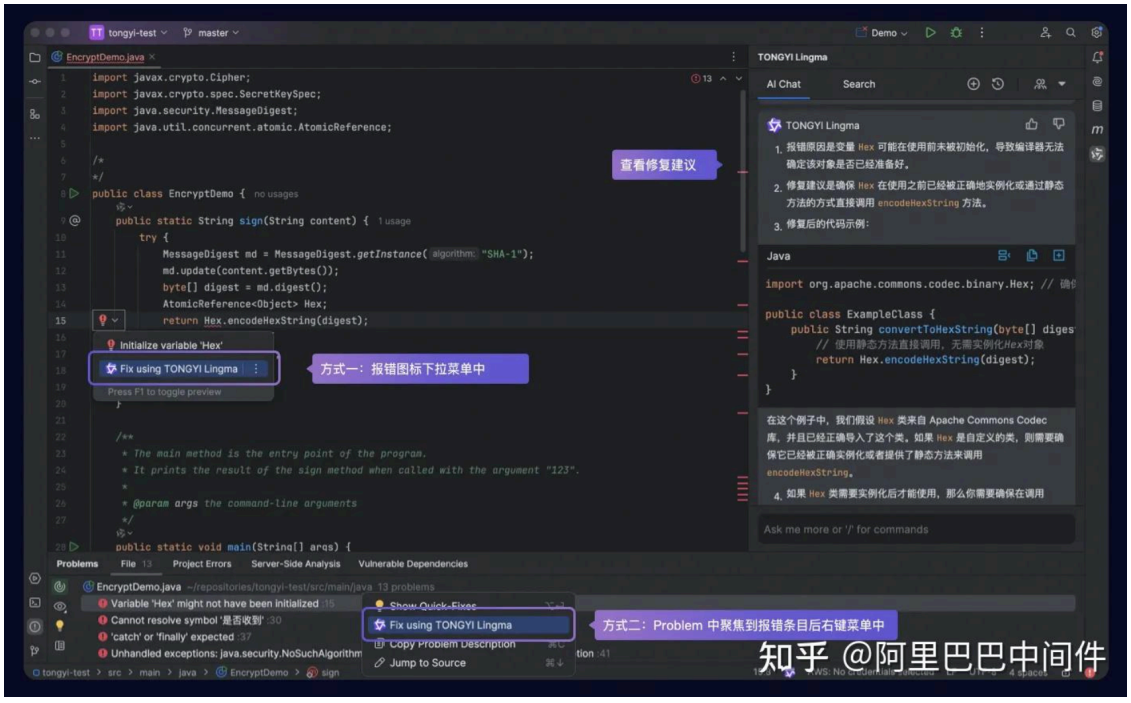
- AI美颜相机App（如美图秀秀），自动实时美颜、瘦脸、滤镜优化；
- 手机输入法的语音识别与智能联想（如讯飞输入法、百度输入法）；
- 电商平台的商品智能推荐（如淘宝、京东首页个性化推荐）；
- 汽车的自动刹车辅助系统（AEB）与自适应巡航；
- 智能家居设备的自动灯光调节与人脸识别解锁。

1.1.2 AI Copilot（AI副驾驶）

（1）定义

AI Copilot 代表人机协作的中间形态，AI在此模式下作为人类的智能助手，与用户并行工作，实时提供建议、辅助分析和生成内容，但最终的决策权始终掌握在人类手中。用户需要主动发起交互，AI则通过聊天窗口或侧边栏界面快速响应，形成一种"并肩作战"的协作关系。

这种协作式交互（Human-in-the-Loop）允许AI在用户输入指令后立即给出针对性建议，例如在代码编写过程中，通义灵码可以实时补全代码块、解释逻辑并提出优化方案，用户可以随时接受、修改或拒绝。这种实时响应能力极大提升了工作效率，同时保留了人类对最终结果的控制权，避免了AI完全自主带来的风险。技术上，AI Copilot 主要依赖LLM + Prompt工程实现，能够快速集成到现有办公或开发工具中。



（2）常见应用：

- 通义灵码在 Pycharm 中的代码自动补全、逻辑解释与修复建议；
- WPS AI在 Word、Excel、演示文稿中的文档生成、数据分析与智能排版；
- 讯飞智作或文心一言的实时写作润色与内容扩展；
- 企业微信或钉钉的AI智能助手（会议纪要、邮件撰写、知识问答）；

1.1.3 AI Embedded 与 AI Copilot的核心区别

（1）区分的核心维度

AI Embedded 与 AI Copilot 的本质区别在于**AI的可见性、交互模式以及人类介入程度。**这一区分并非基于具体功能（如“代码补全”或“美颜”），而是基于AI在用户体验中的角色定位：AI Embedded 是“隐形引擎”，完全融入产品后台；AI Copilot 是“可见助手”，以并行协作方式提供实时建议。

（2）AI Embedded 的本质特征

AI Embedded 的核心是**无感知自动运行。**AI能力被完全嵌入产品底层，成为产品功能不可分割的一部分，用户在日常使用中无法察觉到AI的存在。

- **不可见性**：AI在后台被动触发，无需用户主动调用或交互界面。
- **自动执行**：系统根据预设规则或上下文自动完成任务，用户仅感受到“产品变聪明了”，而非“AI在帮忙”。
- **人类角色**：完全无需介入，AI独立完成单一功能增强。
- **典型场景**：相机App的美颜功能——用户打开相机后，AI自动实时识别人脸、应用瘦脸滤镜、优化肤色，整个过程零交互，用户只看到最终成像效果。这种“消失在产品中”的设计，正是Embedded模式的标志。

（3）AI Copilot 的本质特征

AI Copilot 的核心是**可见协作辅助。**AI以助手身份出现在用户界面中，与用户形成“并肩工作”的关系，用户主动发起交互，AI提供建议，但最终决策权始终由人类掌握。

- **可见性**：AI拥有独立交互界面（如侧边栏、聊天窗口或代码补全提示），用户能明确感知到“AI在给出意

见”。

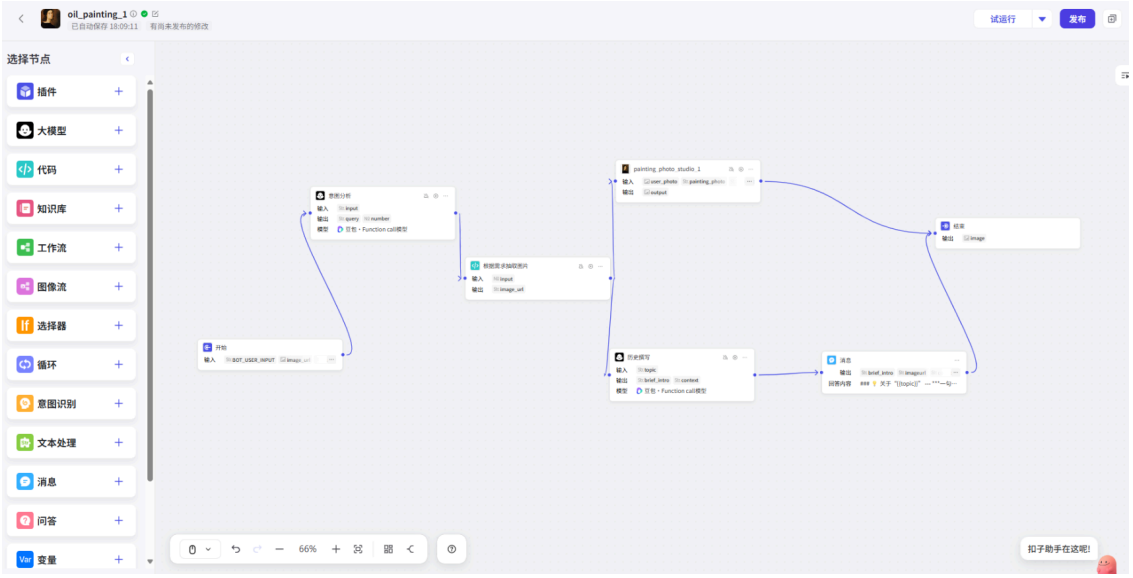
- **实时建议**：AI根据用户当前操作不断提供选项，用户可选择接受、修改或忽略，形成持续反馈循环。
- **人类角色**：决策者（Human-in-the-Loop），AI仅辅助而非替代。
- **典型场景**：通义灵码或WPS AI的代码/文档补全——用户正在编写代码或文档时，AI实时在光标处弹出补全建议或侧边栏分析，用户需主动决定是否采纳。这种“不断给你提意见”的协作模式，正是Copilot的典型特征。

1.1.4 AI Agent（AI智能体）

（1）定义

AI Agent 是三种模式中自主程度最高的形态，AI在此模式下能够完整理解用户给定的目标，自主进行任务规划、工具调用、迭代执行和结果优化，直至任务最终完成。用户仅需提供明确的目标，AI便像一位独立执行者一样开展工作，人类仅在必要时进行最终监督（Human-on-the-Loop）。

其目标驱动（Goal-oriented）的特性体现在AI会主动拆解复杂任务，例如旅行规划Agent会自动查询实时天气、比价机票、生成行程表并处理可能的变更。这种自主规划与多步执行能力基于ReAct循环实现：AI先思考（Thought）、再调用工具（Action）、观察结果（Observation），循环迭代直到目标达成。同时，AI Agent 具备记忆、反思与纠错能力，能够长期维护对话状态并自我优化执行路径。技术核心包括LLM + Tool + Memory + Planner 的完整组合，使其能够处理真实世界的复杂场景。



在实际业务价值上，AI Agent 能够实现端到端自动化，显著降低人力成本，尤其适合重复性高或跨系统协作的任务，例如自主研究报告生成或多Agent团队协作。然而，其挑战也较为突出：自主性越高，幻觉风险和执行偏差越大，因此需要强大的监控、评估和安全机制来保障可靠性。

（2）典型应用

- 自主文献综述Agent（自动检索论文、提取关键信息、生成结构化报告）；
- 代码审查与优化Agent（自动扫描代码仓库、定位Bug、提出重构方案并生成Patch）；
- 财务数据分析Agent（自动拉取多系统数据、生成分析报表并提出异常预警）；
- 销售线索跟进Agent（自动从CRM系统提取线索、生成个性化邮件并跟踪回复）；
- 多Agent协同开发系统（一个Agent负责需求拆解、一个负责代码生成、一个负责自动化测试）。

1.1.5 三者对比

三种模式共同构成AI应用从“隐形增强”到“协作辅助”再到“自主执行”的完整演进路径。AI Embedded 侧重产品体验的无缝提升，AI Copilot 强调效率倍增的人机协作，而AI Agent 则实现真正的任务自动化。在实际项目中，企业往往先采用Embedded模式打基础，再引入Copilot提升日常效率，最终通过Agent实现核心流程的智能化转型。

如果我们拿职场上的角色来类比，那么三者可以分别类比成

技术类型	类比角色	能力描述	独立性	示例任务
Embedded LLM	外包团队	无法随意干预过程，只能依赖他们提供的接口或结果。	低	图像自动美颜
Copilot	助理/秘书	辅助人完成日常工作，可生成文档、代码、报告，依赖指令指导	中	代码补全，文档补全，语音转换文字
Agent	专员/正式员工	独立执行任务，可调用工具、做多步决策、处理动态信息	高	自动查询天气/股票、分析数据、生成报告并推送通知

1.2 技术模块

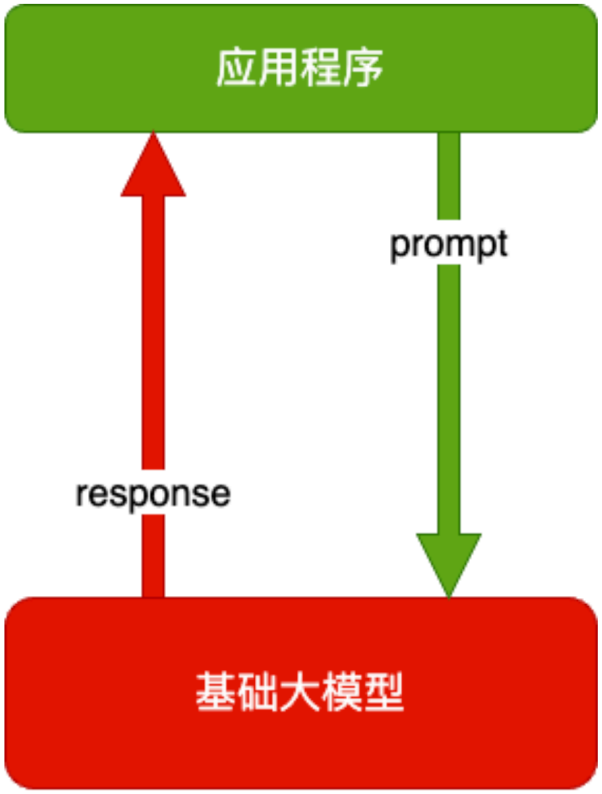
技术模块式是构建三种应用模式的基础工具与实现手段，服务于整个大模型应用架构的不同层次，类似于“砖块、钢筋和水泥”，而非“房子本身”。 任何一种应用模式（Embedded、Copilot或Agent）都可能同时使用这些技术，只是使用方式、组合深度和暴露程度不同。

1.2.1 直接调用大模型（LLM API）

直接调用大模型是最基础的技术模块。

它指通过API接口直接向LLM发送Prompt并获取文本输出，整个过程完全被动，仅依赖模型内置的训练知识完成一次性的生成任务。这种调用方式在大模型应用架构中位于基础设施层和应用接口层，是所有其他模块的起点。

它的优势在于实现简单、延迟低，适合那些不需要外部数据或工具的场景，例如在AI Embedded模式中作为后台隐形引擎运行，或者在AI Copilot的早期原型中提供基础对话能力。然而，由于模型知识存在截止日期且无法主动获取实时信息或执行外部操作，这种直接调用方式的边界非常明显，在处理动态任务时容易出现幻觉或信息过时的问题，因此它通常需要与其他模块结合才能支撑更复杂的应用。

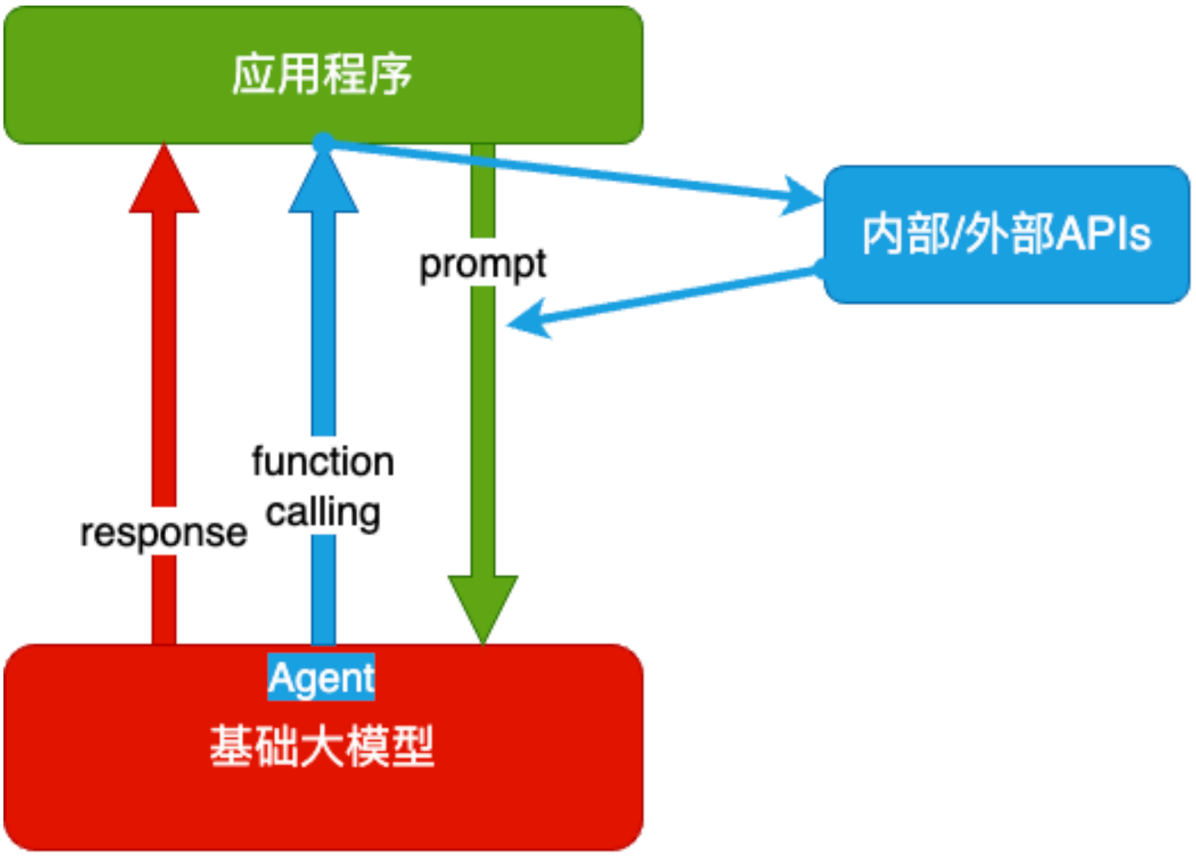


1.2.2 Function Call

Function Call（也称为工具调用或Plugin）是赋予大模型“动手能力”的关键技术模块。

它允许LLM不再仅仅输出最终文本，而是先输出一个结构化的JSON格式函数调用请求，由系统执行外部工具后再将结果反馈给模型，形成 ReAct 式的思考-行动-观察循环。这一模块在大模型应用架构中明确属于工具与能力层，是从“只会说话”向“会做事”跃迁的核心使能技术。

在实际应用中，Function Call既可以作为AI Copilot的可见辅助组件（例如通义灵码在代码编辑时实时调用补全工具），也可以深度融入AI Agent的自主规划流程中（例如旅行规划Agent自动调用天气API和订票接口）。与直接调用相比，Function Call显著提升了系统的外部交互能力，但同时也引入了工具执行的可靠性、成本控制和安全防护等新的治理需求。



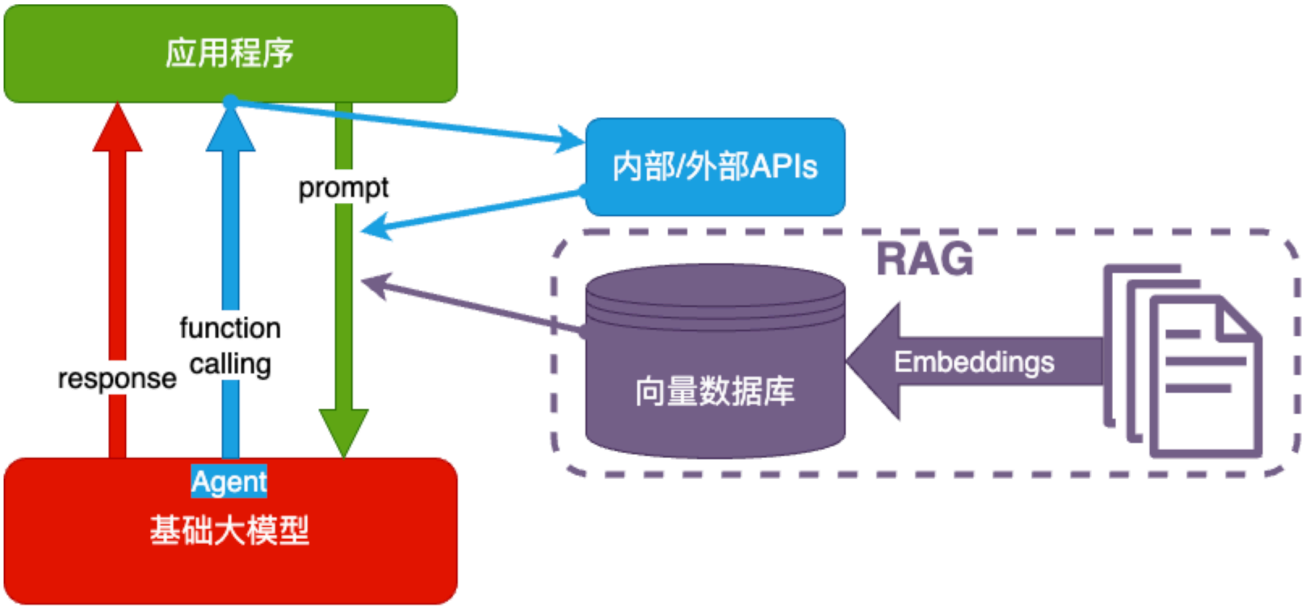
1.2.3 RAG

RAG（Retrieval-Augmented Generation，检索增强生成）则是解决LLM知识静态性问题的核心技术模块。

它通过向量数据库将企业内部文档、知识库或外部数据预先Embedding并存储，当用户提问时，先进行向量检索获取最相关片段，再将这些片段注入Prompt上下文，从而让模型生成更准确、更有针对性的回答。RAG在大模型应用架构中对应数据与记忆层，是实现长期知识注入和个性化响应的基础手段。

在AI Embedded模式中，RAG常常以无感知方式运行，例如电商推荐系统自动从商品库中检索最新信息并嵌入生成结果；在AI Copilot和AI Agent中，RAG则进一步与记忆模块结合，支持更长的对话历史和领域知识增强。

与Function Call不同，RAG侧重于“知识补充”而非“动作执行”，它无法处理实时动态数据（如当前天气或股价），但在处理企业内部文档查询或专业领域问答时表现出色，是降低幻觉率、提升回答可靠性的重要手段。



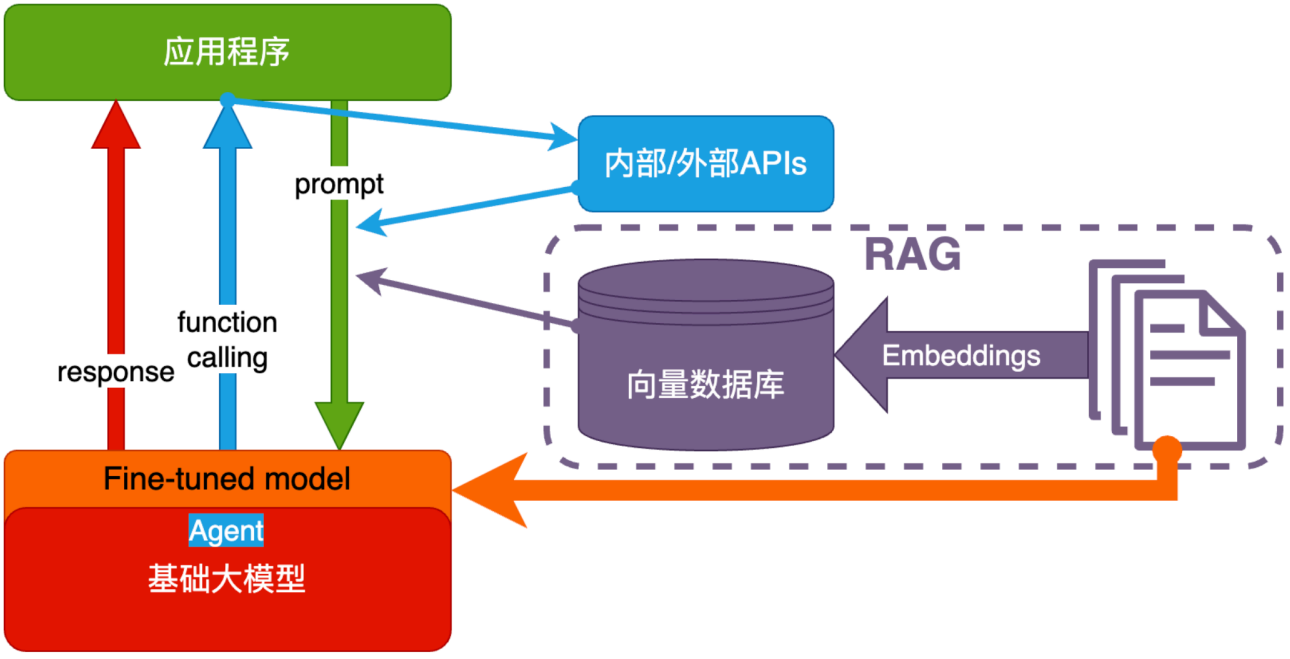
1.2.4 Fine-tuned

Fine-tuned（模型微调）是四个模块中唯一涉及模型参数调整的技术模块。

它通过在特定领域的数据集上对预训练大模型进行进一步训练，使模型在目标任务上获得更高的专业性和准确性，同时减少通用幻觉。Fine-tuned在大模型应用架构中主要位于基础设施层（自定义模型部署），有时也会与数据层结合使用。

它与前三个模块的最大区别在于：前三者主要通过外部机制扩展能力，而Fine-tuned则是从模型内部进行定制化优化。在实际落地中，Fine-tuned常用于构建高精度AI Embedded（如医疗影像诊断系统的专用模型），也可以作为AI Copilot或AI Agent的基础大脑（例如一个经过企业知识微调的代码生成模型）。

相比直接调用，Fine-tuned显著提升了领域适应性，但需要较高的计算资源和高质量标注数据，且模型一旦微调完成，后续更新迭代的成本相对较高。



2. 能力边界拓展

2.1 Function Call

2.1.1 技术演进

（1）直接调用

直接调用大模型是构建AI应用的最基础方式。它通过API接口向LLM发送Prompt，模型根据训练数据直接生成文本回复，整个过程完全被动且一次性完成。这种调用方式实现简单、延迟低，适合处理静态知识问答或通用文本生成任务。然而，由于LLM的知识来源于训练数据存在固定截止日期，同时模型本身无法主动访问外部世界，直接调用方式在实际应用中面临两大核心局限：一是知识过时，无法回答当前实时信息；二是能力边界有限，无法执行外部操作或复杂计算。

（2）RAG

为了解决知识过时以及内部知识的问题，技术演进到了RAG（Retrieval-Augmented Generation，检索增强生成）。

RAG通过向量数据库将企业内部文档、知识库或外部资料预先切块并Embedding存储。当用户提问时，系统先进行向量检索，获取最相关的文档片段，再将这些片段注入Prompt上下文，让LLM基于增强后的知识生成答案。RAG有效解决了LLM知识滞后性问题，使模型能够准确回答企业内部政策、产品手册、历史文档等静态领域内容，显著降低了幻觉率，提升了回答的专业性和可靠性。

（3）RAG的问题

尽管RAG解决了知识过时问题，但它仍然存在明显局限。

首先，RAG只能检索已预存入知识库的静态数据，无法获取实时动态信息，例如当前天气、最新股价、航班余票或实时新闻。

其次，RAG无法让模型主动执行外部操作，如读写文件、调用数据库、发送邮件或运行代码计算。

它本质上仍是“被动增强知识”，模型仍然只能输出文本，无法真正“动手做事”。这些局限导致在需要实时数据或多步外部交互的场景中，单纯依赖RAG的系统会迅速遇到瓶颈。

（4）Function Call

Function Call（工具调用，也称为Plugin或Tool Use）正是为了突破上述RAG局限而引入的关键技术模块。

它赋予LLM主动调用外部工具的能力：模型不再直接输出最终答案，而是先输出一个结构化的JSON函数调用请求，系统执行工具后将结果（Observation）返回给模型，模型再基于新信息继续思考，形成 ReAct 循环。

这一机制让LLM真正具备了“手和眼”，实现了从“只会说话”到“会做事”的本质跃迁。Function Call在大模型应用架构中属于工具与能力层，它不仅能解决实时数据获取问题，还能支持文件读写、数据库操作、API调用等复杂外部交互，从而使AI系统具备真正的执行力。

2.1.2 实现逻辑

（1）定义

Function Call（函数调用，也称为工具调用或Function Calling） 是大模型应用架构中工具与能力层的核心机制。

这一机制的核心设计理念在于严格的权限隔离与受控能力扩展。模型本身被置于一个安全的沙箱环境中，永远无法直接访问或操作外部系统（如文件系统、数据库、网络API、代码执行环境等）。它只能通过预先注册的标准接口提出**“调用请求”**，由可信的外部运行时系统来实际执行操作。这种“模型只提需求、系统负责执行”的设计，既保障了系统的安全性、可审计性和可控性，又让LLM真正具备了超越纯文本生成的能力。

（2）调用请求标准格式

Function Call 的调用请求采用**统一的结构化 JSON 格式**，这是目前业界（OpenAI、Anthropic、Google、LangChain、Coze、Dify 等主流平台）共同遵循的标准规范。

```
{
  "name": "函数名称",
  "arguments": {
    "参数名1": 参数值1,
    "参数名2": 参数值2,
    ...
  }
}
```

字段说明：

- **name**（必填，字符串）：要调用的工具函数的名称，必须与提前注册到模型中的 Tool 名称完全一致。
- **arguments**（必填，对象）：传递给函数的具体参数，以键值对形式存在。参数的名称和类型必须严格匹配 Tool 的定义，否则系统会拒绝执行。

2.1.3 示例

现在我们想让大模型获得读取本地文件的能力，我们应该如何设计呢?

（1）读取本地文件的函数

```
def read_file(file_path)
    with open(file_path,'r',encoding = 'utf-8') as f:
        content = f.read()
    return content
print(read_file('./data.txt'))
```

（2）大模型调用

```
from dotenv import load_dotenv
load_dotenv()
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model_name = 'deepseek-chat')

res = llm.invoke('''
请执行下面的代码：
with open('./data.txt','r',encoding = 'utf-8') as f:
    content = f.read()
''')
print(res.content)
```

显示结果：

```
我无法直接在你的本地计算机上执行代码或访问文件系统，因为我运行在云端环境中，没有权限访问你的本地文件（如 `./data.txt`）。

不过，我可以帮你做以下事情：

1. **解释代码**：这段代码的作用是打开当前目录下的 `data.txt` 文件，以只读模式（`'r'`）并指定 UTF-8 编码读取全部内容，然后储存在变量 `content` 中。

2. **模拟运行**：如果你能提供文件的内容，我可以帮你模拟处理数据。

3. **提供替代方案**：如果你希望我在我的沙箱环境中创建一个测试文件并执行类似操作，我可以这样做。

你想要哪种帮助？比如：
- 你能否分享一下 `data.txt` 的内容？
- 或者告诉我你希望从文件中提取或处理什么样的信息？
```

(3) 直接拿调用指令?

```
prompt = '''
函数：
with open('./data.txt','r',encoding = 'utf-8') as f:
    content = f.read()
现在按照以下JSON结构调用函数：
{
  "name": "read_file",
  "arguments": {
    "file_path": "文件路径"
  }
}
'''
res = llm.invoke(prompt)
print(res.content)
```

显示结果:

```
{"name": "read_file", "arguments": {"file_path": "./data.txt"}}
```

(4) 按照标准封装格式?

```
tool = '''{
  "type": "function",
  "function": {
    "name": "read_file",
    "description": "读取指定本地文件的内容并返回完整文本字符串。适用于查看代码文件、配置文件、日志文件、数据文件等场景。",
    "parameters": {
      "type": "object",
      "properties": {
        "file_path": {
          "type": "string",
```

```
            "description": "要读取的文件路径，支持相对路径或绝对路径，例如
'./data.txt'、'requirements.txt'、'/app/config.json'"
        }
      },
      "required": ["file_path"],
      "additionalProperties": False
    }
  }
} '''
```

（5）实际调用

```
prompt = '''
你是一个专业且可靠的AI助手，能够帮助用户读取本地文件并对文件内容进行分析和总结。

你拥有以下工具：
{tool}


使用工具的规则：
1. 当用户要求读取某个文件、查看文件内容、分析文件，或你需要文件内容才能回答问题时，必须使用工具。
2. 不要直接猜测或编造文件内容，必须先调用工具获取真实内容。
3. 调用工具时，只输出工具调用JSON，不要添加任何额外文字或解释。
4. 工具调用格式必须严格按照以下JSON结构（不要转义参数，不要添加markdown代码块）：
{{
  "name": "read_file",
  "arguments": {{
    "file_path": "./data.txt"
  }}
}}
5. 收到工具返回的结果（Observation）后，基于真实文件内容给出自然、准确的回答。
6. 如果读取失败，请如实告知用户。

用户要求：
{query}
'''
query = "需要打开的文件位置：  ./data.txt"
final_prompt = prompt.format(tool=tool,query=query)
res = llm.invoke(final_prompt)
print(res.content)
```

生成结果：

```
{"name": "read_file", "arguments": {"file_path": "./data.txt"}}
```

我们发现，不管在提示词里面如何写，大模型本身并不具备打开本地文件的能力。

- **模型不会自己运行 Python**，Function Call 只是**让模型生成调用指令。**
- **实际函数执行必须由系统端完成。**

那么我们可以从输出调用指令本身下手。

2.1.4 调用指令实行

那么我们考虑另外一方面，能不够能根据大模型输出的调用指令，我们写代码在 python 里面实际调用?

（1）实行代码

```python
def read_file(file_path):
    """读取文件内容并返回"""
    with open(file_path, 'r', encoding='utf-8') as f:
        content = f.read()
    return content

# 函数注册表
functions = {
    "read_file": read_file
}

# 调用函数
json_input = {
    "name": "read_file",
    "arguments": {
        "file_path": "./data.txt"
    }
}

# 根据JSON结构执行函数调用
if json_input["name"] in functions:
    result = functions[json_input["name"]](**json_input["arguments"])
    print(result)
else:
    print(f"函数 {json_input['name']} 未找到")
```

（2）能力边界拓展

大模型（LLM）本质上是一个**纯文本生成器。**在没有 Function Call 之前，它只能“说话”——输出一段文字、代码、建议或回答，却**无法真正对外部世界产生任何实际影响。**

当我们加入 Function Call 后，情况发生了本质变化：

- **大模型 = 大脑**（负责思考、规划、决策）
- **Function Call + Tool = 一只手臂**（负责真正去执行动作）

大脑只能思考，但有了手臂才能真正“做事”——伸手拿东西、操作工具、改变外部环境。

（3）与“直接调用函数”的本质区别

区别在于**控制权**：

直接调用函数（开发者写死）：

- 控制权在**开发者**手里。
- 什么时候调用、调用什么函数、用什么参数，都是开发者提前写死的。
- 模型只是一个“回答器”，它无法决定要不要调用、调用几次、调用顺序。
- 适合固定流程，但无法处理动态、复杂、多步的任务。

Function Call（模型生成调用指令）：

- 控制权在**大模型**手里。
- 模型自己判断：当前是否需要调用工具？调用哪个工具？用什么参数？
- 只有当模型认为有必要时，才会生成调用指令。
- 系统只负责"执行模型下达的指令"，模型才是真正的"指挥官"。

这正是 Agent 的核心：**模型成为主动的执行者，而不是被动的回答者。**

2.2 接口

接口（Interface）是不同系统、组件或设备之间进行**通信和数据交换的标准化规范。**它定义了"双方如何对话、如何传递信息、如何协同工作"的规则。

简单来说：**接口就是"连接器"**，它让两个原本独立的系统能够相互理解并配合完成任务。



2.2.1 接口的进化

接口演化的核心趋势是：越来越适应人的习惯，越来越自然。 从早期"让人类适应机器"，逐步演变为"让机器适应人类"，最终目标是实现人机之间几乎无摩擦的交互。

（1）命令行接口（CLI）

这是最早的接口形式，以文字命令为主。用户必须输入精确的指令（如 dir、ls、cp），计算机才能执行操作。

- 特点：高效、占用资源少，但学习成本高
- 代表：DOS、Unix/Linux Shell、Windows PowerShell
- 局限：需要记忆大量命令，普通用户难以上手

（2）图形用户接口（GUI）

Graphical User Interface（图形用户接口）

通过图形、图标、窗口、鼠标操作来交互，大幅降低了使用门槛。

- 特点：所见即所得、直观易用
- 代表：Windows、macOS、iOS、Android
- 优势：极大普及了计算机，让普通人也能轻松操作
- 局限：依然需要手动点击、拖拽，操作步骤较多

（3）语言接口 / 会话接口（CUI / LUI / NLI）

Conversational User Interface / Natural-Language User Interface（会话式 / 自然语言接口）← **我们当前所处的位置**

这一阶段，人类终于可以用**日常自然语言**（中文、英文）直接与系统对话，系统理解意图后执行操作。

- 特点：零学习成本、支持模糊表达、支持多轮对话
- 代表：Siri、小爱同学、ChatGPT、我们正在使用的 Function Call
- **核心意义**：Function Call 正是这一阶段的典型实现方式。 我们用自然语言说“读取 ./data.txt 文件”，大模型理解意图后，自动生成结构化的调用指令（JSON），系统再执行对应的 Tool。这整个过程**本质上就是一种新型的接口调用**——以自然语言为载体、以 Function Call 为协议的 Natural Language Interface（NLI）。

（4）脑机接口（BCI）

Brain-Computer Interface（脑机接口）

这是接口演化的最终方向，人类无需说话、无需打字、无需点击，直接通过大脑意念与机器进行交互。

- 特点：最自然、无介质、无延迟
- 代表：Neuralink、脑电波控制设备
- 未来意义：人机真正实现“心想事成”，接口将彻底消失在感知之外

2.2.2 MCP（Model Context Protocol）

MCP 是自然语言接口（NLI）发展到当前阶段的重要里程碑。它由 Anthropic 于 2024 年底提出并开源，被业界称为 “AI 的 USB-C 接口”。

（1）MCP 的背景

随着 Function Call 的广泛应用，一个明显的问题逐渐暴露出来：**碎片化严重。**

不同大模型平台（OpenAI、Claude、通义千问、Grok 等）和不同 Agent 框架（LangChain、Dify、Coze 等）对 Tool 的定义方式、调用格式、参数描述、安全控制都不相同。这导致开发者需要为每个平台重复编写适配代码，维护成本极高，也严重阻碍了 AI Agent 的大规模落地。

为了解决这一碎片化问题，Anthropic 推出了 **MCP（Model Context Protocol，模型上下文协议）**，旨在为大模型与外部工具、数据源、设备之间的交互提供一套**统一、开放、可互操作的标准协议。**

（2）MCP 的核心价值

MCP 的核心价值在于**标准化与可复用性**，主要体现在以下四个方面：

- **工具发现**：模型可以自动查询 Server 端提供了哪些 Tool，无需提前硬编码。
- **统一调用规范**：提供标准化的 Tool 描述、调用格式、参数验证和返回结构。
- **安全与权限控制**：内置细粒度权限管理、审计日志和沙箱机制，大幅提升生产环境安全性。
- **跨平台互操作**：一次开发，可被多种 AI 客户端（Claude、Cursor、VS Code 等）通用，极大降低开发和维护成本。

MCP 的出现，让 Function Call 从“各家各玩各的”走向了行业统一标准，为构建复杂、多平台兼容的 AI Agent 系统奠定了基础。

（3）MCP 的本质

MCP 本质上仍然是 Function Call 的标准化升级版，而不是一种全新的、完全不同的机制。

- 它**没有改变**“模型生成调用指令 → 系统执行 Tool → 返回结果给模型”这一核心工作流程。

- 它只是把以前**非标准、碎片化的 Function Call**，统一成了一个开放的、规范的协议。
- 可以理解为：**Function Call 是“打电话”这个行为本身**，而 **MCP 是统一的“电话通信协议”**（比如从模拟电话升级到数字统一标准）。

因此，学习 MCP 的最佳路径是：**先掌握 Function Call 的原理，再学习 MCP 的标准化实现。**两者并不冲突，而是层层递进的关系。

MCP 不是要取代 Function Call，而是让 Function Call 变得更标准、更易用、更适合生产环境。我们先把 Function Call 的核心原理学扎实，再理解 MCP 的标准化升级，就会非常自然和顺畅。

2.2.3 Tool

在前面我们已经详细讲解了 **Function Call** 的机制：模型不再直接输出最终答案，而是生成结构化的调用指令（JSON），让系统去执行外部操作。

而真正让这个机制落地的、**让大模型能力边界发生本质扩展的核心**，正是 **Tool。**

（1）Tool 的定义

Tool（工具） 是大模型能够实际调用并执行的**具体函数或代码实现。** 它是大模型与外部世界进行真实交互的**执行体。**

简单来说：

- Function Call = 大脑发出的“指令”（模型生成 JSON）
- **Tool = 真正执行指令的“手臂”**（开发者编写的可被调用的函数）

Tool 才是大模型从“只会说话”真正走向“能做事”的关键载体。

（2）为什么 Tool 是大模型能力边界扩展的最核心演进？

在没有 Tool 之前，大模型的能力边界被严格限制在**纯文本生成**范围内：

- 只能基于训练数据回答问题
- 无法获取实时信息
- 无法操作外部系统（读写文件、调用 API、执行代码、操作数据库等）

当我们引入 Tool 之后，大模型的能力边界发生了**质的突破**：

- **实时性**：可以调用天气 API、股票接口、搜索工具获取最新数据
- **执行力**：可以读写文件、发送邮件、操作数据库、运行代码
- **专业性**：可以调用数学计算工具、图像处理工具、特定领域 API
- **复杂性**：通过多个 Tool 的组合，实现多步规划和复杂任务

Tool 的出现，标志着大模型从“知识型 AI”走向了“行动型 AI”，这是 Agent 能够真正落地的最核心基础。

（3）Tool 与 Function Call 的关系

- **Function Call** 是**调用协议**（模型如何发出指令）
- **Tool** 是**执行实体**（真正被调用的函数）

两者缺一不可：

- 只有 Function Call 没有 Tool，模型只会“喊口号”，无法真正做事；
- 只有 Tool 没有 Function Call，模型无法自主决定何时调用、如何调用。

Tool + Function Call 共同构成了大模型能力边界扩展的完整闭环。

2.3 Tools 的实现

2.3.1 项目背景

在实际开发大模型应用的过程中，我们经常需要让模型了解输入或输出文本的 Token 数量（例如用于成本控制、上下文长度管理、计费等场景）。

然而，如果我们**直接让大模型自行统计 Token 数量**，结果往往并不准确。

主要原因如下：

- 大模型本身**没有内置 tokenizer**，它只负责生成文本，无法精确知道每个字符对应的 Token 数量；
- 不同模型（GPT-4o、Claude、通义千问等）使用的 tokenizer 规则完全不同；
- 如果让模型“估算”Token 数，它会产生较大误差，甚至给出完全错误的结果。

为了解决这一问题，我们决定**为大模型提供一个专用的 Tool —— count_tokens**，让模型在需要时调用这个工具来完成精确的 Token 统计。

这个 Tool 的实现，正是我们理解和掌握 Tool 机制的一个典型切入点。它清晰地展示了 **Tool 的核心价值**：帮助大模型突破自身能力边界，完成它原本无法直接完成的任务。

2.3.2 Tools 的标准写法

在实际开发中，Tool 的写法需要遵循规范，这样大模型才能正确理解和调用。

（1）**推荐的标准写法**（LangChain 官方推荐风格）：

```python
from langchain.tools import tool

@tool
def count_tokens(text: str, model: str = "gpt-4o") -> int:
    """
    精确统计一段文本在指定模型下的 Token 数量。

    参数:
        text: 需要统计的文本内容
        model: 使用的模型名称（默认 gpt-4o）

    返回:
        该文本对应的 Token 数量（整数）
    """
    # 具体实现代码（见下一小节）
    ...
```

（2）**重点说明 `@tool` 装饰器**：

- `@tool` 是 **LangChain 提供的一个装饰器。**
- 它的作用是：**把一个普通的 Python 函数，自动转换成 LangChain 可以识别和使用的 Tool。**
- 加上 `@tool` 后，LangChain 会自动完成以下工作：
    - 自动生成完整的 JSON Schema（模型需要的工具描述）
    - 自动提取函数名、描述、参数信息
    - 自动把函数包装成 Tool 对象
- 注意：`"""..."""` 这段文字描述内容，一定要保留，并且要描述清楚，这段内容是在想大模型介绍你的这个函数的功能。

2.3.3 函数定义

下面是 `count_tokens` Tool 的完整规范实现：

```python
import tiktoken
from langchain.tools import tool

@tool
def count_tokens(text):
    """
    精确统计一段文本的 Token 数量（使用 cl100k_base 编码）。

    参数:
        text: 需要统计 Token 数量的文本内容

    返回:
        该文本对应的 Token 数量（整数）
    """
    # 使用 tiktoken 精确计算
    enc = tiktoken.get_encoding("cl100k_base")
    tokens = enc.encode(text)
    return len(tokens)
```

测试是否注册成功：

```python
# 测试 Tool 是否注册成功
print(count_tokens.name)           # 输出: count_tokens
print(count_tokens.description)    # 输出工具描述
```

2.3.4 工具绑定

在前面我们已经定义好了 Tool，现在需要把 Tool “告诉”大模型，让模型知道自己拥有哪些工具。

这里用到的关键方法就是 `bind_tools`。

（1）`bind_tools` 的定义

```python
llm_with_tools = llm.bind_tools(tools)
```

- `bind_tools` 是 LangChain 中 Chat 模型（`ChatOpenAI`、`ChatClaude` 等）提供的一个**重要方法。**
- 它的作用是：**将 Tool 绑定到大模型上**，让模型在思考时知道自己可以使用哪些工具。

（2） `bind_tools` 的核心作用

- **让模型"知道"自己拥有工具**

绑定之后，模型在生成回复时，会自动判断是否需要调用工具，并按照 Tool 的 Schema 生成标准的 Function Call（JSON）。

- **自动生成工具调用指令**

模型不再只是输出普通文本，而是会在需要时输出 `tool_calls`（工具调用列表）。

- **简化开发流程**

开发者无需手动拼接提示词或 Schema，LangChain 会自动处理 Tool 的描述信息并注入到模型的调用参数中。

（3）接下来我们让大模型发出function_call指令

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from langchain_core.
# 1. 准备 LLM 和 Tool
llm = ChatOpenAI(temperature=0)
tools = [count_tokens]                    # 把 Tool 放入工具池

# 2. 把 Tool 绑定到 LLM（关键一步）
llm_with_tools = llm.bind_tools(tools)     # 模型现在知道自己拥有 count_tokens 这个 Tool

# 3. 用户输入
user_input = "帮我统计下面这段话的 Token 数：人工智能的发展正在深刻改变我们的生活方式和生产方式。"

# 4. 让模型生成调用指令（Function Call）
messages = [HumanMessage(content=user_input)]
response = llm_with_tools.invoke(messages)

print("=== 模型输出的 Function Call ===")
print(response.tool_calls)   # 这里就是模型生成的调用指令
```

模型此时会输出类似以下内容（Function Call JSON）：

```
[{'name': 'count_tokens', 'args': {'text': '人工智能的发展正在深刻改变我们的生活方式和生产方式。'}, 'id': 'call_OO_yxaAWoXJ4CKT2zLHAxRB3445', 'type': 'tool_call'}]
```

2.3.5 系统执行 Tool

```python
# 最终落地
if response.tool_calls:
    tool_call = response.tool_calls[0]
    tool_name = tool_call["name"]
    tool_args = tool_call["args"]

    if tool_name == "count_tokens":
        result = count_tokens.invoke(tool_args)   # 真正执行 read_file / count_tokens
        print("Tool 执行结果：", result)
```

2.4 调用外部 API

在实际的大模型应用开发中，我们经常需要让AI获取**实时动态数据**，例如当前天气、股票行情、最新新闻、汇率、航班信息等。这些数据都是不断变化的，无法预先存储在大模型的训练数据中。

问题： 如果我们直接让大模型回答“北京今天的天气怎么样？”，它能否给出准确答案？

```python
print(llm.invoke("现在北京的天气怎么样？"))
```

我们发现无法获取实时知识，主要原因在于：

- 大模型的知识截止于训练数据，无法获取互联网上的**实时信息**；
- 它没有权限直接访问外部网络、数据库或第三方服务；
- 直接让模型“猜测”或“编造”实时数据，会导致严重的幻觉问题，答案不可信。

解决方案： 我们为大模型提供一个**外部API调用 Tool**（例如知心天气API、百度地图天气API等），让模型在需要实时数据时，主动调用该Tool去请求外部接口，获取最新信息后，再基于真实数据给出准确回答。

通过这个项目，我们要实现的是：**让大模型突破“知识静态”的限制，具备实时获取外部数据的能力。**

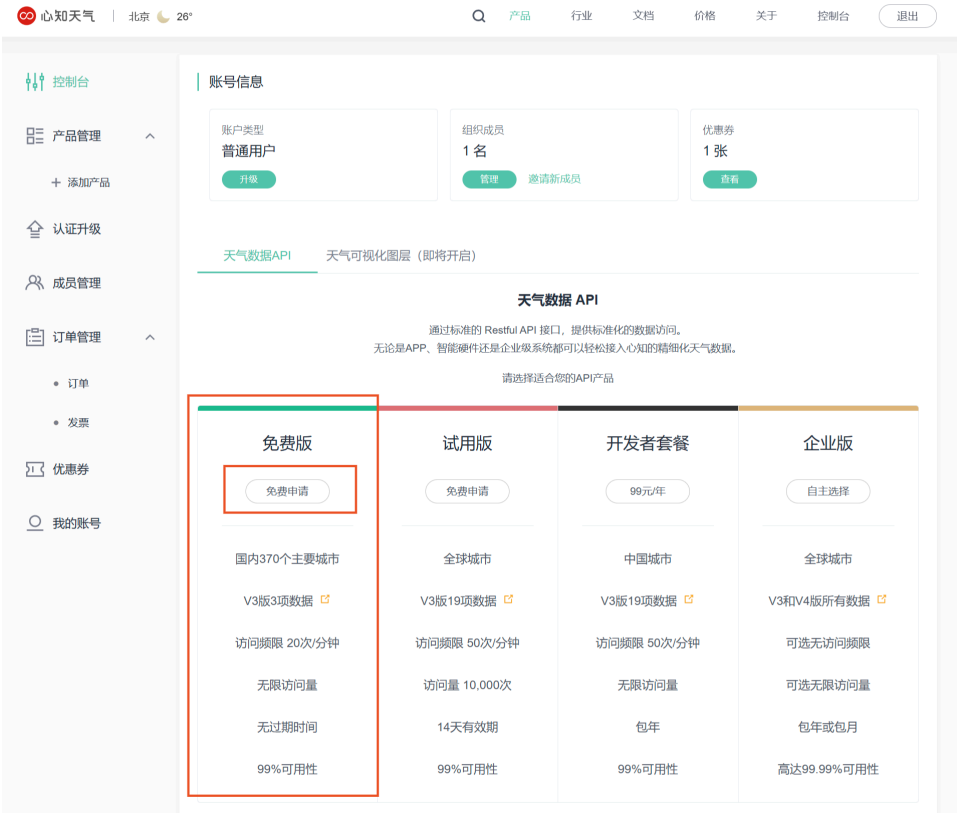
2.4.1 天气调用工具

使用 **心知天气 API**（免费注册，国内稳定访问）

- 注册地址：https://www.seniverse.com/



- 注册完成后，点击免费申请



- 点击进入我的产品



- 获得公钥和密钥



2.4.2 原生态 requests：

```python
import requests
import tkinter as tk

api_key = "your_api_key"#改为自己的api
city_id = "北京"

# 查询天气信息
url = f"https://api.seniverse.com/v3/weather/now.json?key={api_key}&location={city_id}&language=zh-Hans&unit=c&fields=location,update_time"
response = requests.get(url)  #  <-- 最重要的一句话
data = response.json()
print(data)


```

```python
# 检查是否成功获取数据
if "results" in data:
    weather = data["results"][0]["now"]["text"]
    temperature = data["results"][0]["now"]["temperature"]
    location = data["results"][0]["location"]["name"]
    update_time = data["results"][0]["last_update"]

    # 创建窗口
    root = tk.Tk()
    root.title("Weather Information")

    # 显示天气信息
    label_weather = tk.Label(root, text=f"Weather: {weather}")
    label_weather.pack()

    label_temperature = tk.Label(root, text=f"Temperature: {temperature}°C")
    label_temperature.pack()

    label_location = tk.Label(root, text=f"Location: {location}")
    label_location.pack()

    label_update_time = tk.Label(root, text=f"Last Update: {update_time}")
    label_update_time.pack()

    root.mainloop()
else:
    print("Error: Unable to retrieve weather information.")
```

2.4.3 对内容进行封装：

```python
import requests

def get_weather(city: str):
    """
    获取指定城市的实时天气信息

    参数:
        city: 城市名称(中文或拼音)，如"北京"或"beijing"

    返回:
        格式化的天气信息字符串，包含地点、天气、温度和更新时间
        如果查询失败，返回以"错误信息:"开头的字符串
    """
    api_key = "your_api_key"  # 替换为自己的API密钥
    base_url = "https://api.seniverse.com/v3/weather/now.json"

    params = {
        "key": api_key,
        "location": city,
        "language": "zh-Hans",
        "unit": "c"
```

```
    }

    try:
        response = requests.get(base_url, params=params)
        response.raise_for_status()
        data = response.json()

        if "results" in data and data["results"]:
            result = data["results"][0]
            return (
                f"地点: {result['location']['name']}\n"
                f"天气: {result['now']['text']}\n"
                f"温度: {result['now']['temperature']}°C\n"
                f"更新时间: {result['last_update']}"
            )
        return "错误信息: 未找到该城市的天气信息"

    except requests.exceptions.RequestException as e:
        return f"错误信息: 网络请求失败 - {str(e)}"
    except (KeyError, IndexError, ValueError) as e:
        return f"错误信息: 数据解析错误 - {str(e)}"
    except Exception as e:
        return f"错误信息: 未知错误 - {str(e)}"


# 使用示例
if __name__ == "__main__":
    # 查询当前天气
    print(get_weather("北京"))
    print("\n" + get_weather("上海"))

    # 查询失败示例
    print("\n" + get_weather("不存在的城市"))
```

2.4.4 API集合地

国内常用的API网站可以从两大类来看

- 一类是提供丰富接口的**聚合平台**，适合项目开发、功能测试、快速原型搭建；
- 另一类是服务商或软件厂家的**官方开放平台**，适合需要特定功能或稳定企业级服务的场景。

（1）一站式API聚合市场

这是查找和使用各种API最便捷的渠道，汇集了大量来自不同服务商的接口，并提供统一的调用和测试工具。

- **幂简集成 (API HUB)**：**接口覆盖面极广**，汇总了全网超过 **5000** 个API接口和 **1500**+服务商。
- **聚合数据 (Juhe Data)**：**数据质量高，稳定可靠**，拥有超过 **760** 个专有API，提供每日免费调用额度（如天气API每日500次）。
- **阿里云 API 市场**：与**阿里云生态深度集成**，涵盖人工智能、大数据、物联网等官方与第三方高质量服务。
- **百度智能云 API 商城**：**AI能力突出**，提供了大量AI相关的API（如语音技术、图像识别）及基础服务（如

短信、OCR）。

- **APISpace**：提供了天气、快递、实名认证、金融科技、交通地理等丰富的API服务。
- **京东万象数据平台**：**电商数据是其特色**，适合需要商品详情、订单管理等电商相关API的场景。
- **腾讯云 API 中心**：和腾讯云生态紧密结合，提供覆盖计算、存储、数据库、音视频等众多领域的API服务。
- **数据宝**：**国有背景，数据权威合规**，提供身份认证、车辆、运营商等权威数据服务。
- **天眼数聚**：在**企业征信和身份核验**领域有深厚积累，服务已覆盖近万家企业。
- **HaoService**：**生活服务类API是其强项**，提供基站定位、地址解析、坐标服务等。

（2）官方开放平台

如果想直接使用某个大厂或专业机构的数据或功能，直接调用他们的官方API通常更权威、直接。

- **地图类**：国内主要的地图服务商都提供了丰富的API，包括**百度地图API**和**高德地图API**，还有**腾讯位置服务**。
- **电商类**：对于电商开发，官方渠道是首选。可关注**淘宝开放平台 (TOP)**、**拼多多开放平台**和**京东商家开放平台**。
- **AI与语音类**：涉及人工智能领域，可以留意**百度AI开放平台**、**讯飞开放平台**（智能语音）以及**旷视Face++**（人脸识别）。
- **生活服务类**：日常生活中高频使用的功能，如气象服务，可以参考**心知天气**和**和风天气**等专业气象API。

（3） 开发工具与平台

除了查找和调用，还有一些平台专注于**API的调试、测试和管理**，能极大提升开发效率。

- **Apifox**：国产API协作平台，集调试、文档、Mock、测试于一体，被誉为“Swagger+Postman+Mock+Jmeter的结合体”。
- **Eolink**：功能全面的API全生命周期管理平台，支持API开发、测试、网关、监控等企业级需求。
- **ApiPost**：类似Apifox的国产工具，界面简洁，易于上手。

（4） 免费/开源项目

有很多开发者或社区维护的项目，提供了大量免费的API接口，非常适合个人开发者的快速项目原型搭建。

- **YesAPI**：提供数据存储、查询、文件处理、信息验证等多种功能接口。
- **public-apis**：GitHub上非常知名的公共API列表，汇集了全球各种领域的免费API资源。
- **山河API**：提供特定功能接口的API服务项目。
- **ALLBEAPI**：开源库API化平台，开发者无需安装部署，即可通过API调用一些受欢迎的开源库功能。
- **BugPk-Api**和**夏柔API**：一些开发者提供的个人免费接口调用平台，稳定性不一，适合个人项目测试。
- **一些开源免费的接口集**：网络上有很多开发者或个人维护的免费接口项目，内容包罗万象，稳定性较高。

（5）其他可用平台

此外，还有一些平台或工具也可供选择：

- **Apix**：面向金融风控领域，提供身份核验、征信报告等专业API。
- **用友APILink**：偏企业级应用，如企业管理、身份验证、金融风控等。
- **通联数据**：专注于金融大数据，提供行情、研报等金融信息API。

- **极速数据**：提供各类生活数据API。
- **万维易源**：提供API数据交易和服务。

2.4.5 小练习

自己找一个 API 接口，按照上面介绍的方法，封装成 tool，并进行调用。

示例：

```
prompt = '''
北京现在天气怎么样
'''
```

让大模型进行 function_call，并调用API，生成今天的天气